A Mini Project report

On

Heart Beat Sensor Using Arduino

A dissertation Submitted in partial fulfillment of the requirement

for the

Award of the degree of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

By

J.Anil kumar	**21N31A0491**
K.Abhinay Reddy	**21N31A0499**
G.Srinivasa Reddy	**21N31A0473**

Under the esteemed guidance of

DR. P. VANITHA
Associate Professor

DEPARTMENT OF ELECTRONICS & COMMUICATION ENGINEERING

MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY

Autonomous Institution – UGC, Govt. of India

(Affiliated to JNTU, Hyderabad, Approved by AICTE - Accredited by NBA & NAAC – 'A' Grade)

Maisammaguda, Dhulapally (Post Via Hakimpet), Secunderabad – 500100

2021-2025

2021-2025

CERTIFICATE

This is to certify that the **MINI PROJECT** work entitled "**Heart Beat Sensor Using Arduino**" is carried out by **J.Anil Kumar(21N31A0491), K.Abhinay Reddy(21N31A0499), G.Srinivasa Reddy(21N31A0473)**,in partial fulfillment for the award of degree of ***BACHELOR OF TECHNOLOGY*** in Electronics and communication Engineering, Jawaharlal Nehru Technological University ,Hyderabad during the academic year 2024-2025.

INTERNAL GUIDE

Dr.P.Vanitha
Associate Professor

HEAD OF THE DEPARTMENT

Dr K MALLIKARJUNA LINGAM
PROFESSOR & HOD

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the Mini Project titled **"Heart Beat Sensor Using Arduino"** submitted to Malla Reddy College of Engineering and Technology (UGC Autonomous), affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering is a result of original research carried-out in this thesis. It is further declared that the Mini Project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

J.Anil kumar - 21N31A0491

K. Abhinay Reddy- 21N31A0499

G. Srinivasa Reddy-21N31A0473

ACKNOWLEDGMENTS

First and foremost, we would like to thank God for giving strength to complete this work. We feel ourselves honoured and privileged to place our warm salutation to our college Malla Reddy College of Engineering and Technology (UGC-Autonomous) and our Director Dr. VSK Reddy, Principal Dr. S. Srinivasa Rao who gave us the opportunity to have experience in engineering and profound technical knowledge.

We express our heartiest thanks to Prof P. Sanjeeva Reddy, Director of the Department of Electronics and Communication Engineering and Dr.K. Mallikarjuna Lingam, Head of the Department of Electronics and communication engineering for encouraging us in every aspect of our project and helping us realize our full potential.

We would like to thank our Project Coordinator Mrs.Dr.P.Vanitha, Associate Professor, Department of Electronics and Communication Engineering for her valuable guidance and encouragement during my project work.

We would like to thank our internal guide Mrs.Dr.P.Vanitha, Designation, Department of Electronics and Communication Engineering for her regular guidance and constant encouragement. We are extremely grateful to her valuable suggestions and unflinching cooperation throughout project work.

We would like to thank Dr. T. Venugopal, Dean of Academics, who in spite of being busy with his duties took time to guide and keep us on the correct path.

We would also like to thank all the supporting staff of the Department of Electronics and Communication Engineering and all other departments who have been helpful directly or indirectly in making our project a success.

We are extremely grateful to our parents for their blessings and prayers for the completion of our project that gave us strength to do our project.

By

J.Anil Kumar (21N31A0491)
K.Abhinay Reddy (21N31A0499)
G.Srinivasa Reddy(21N31A0473)

CONTENTS

LIST OF FIGURES

ABSTRACT

Heartbeat Sensor is an electronic device that is used to measure the heart rate, i.e. speed of the heartbeat.Monitoring body temperature, heart rate and blood pressure are the basic things that we do in order to keep us healthy. In order to measure the body temperature we use thermometers; and a sphygmomanometer to monitor the Arterial Pressure or Blood Pressure.

Heart Rate can be monitored in two ways: one way is to manually check the pulse either at wrist or neck and the other way is to use a Heartbeat Sensor. In this project, we have designed a Heart Rate Monitor System using Arduino and Heartbeat Sensor. You can find the Principle of Heartbeat Sensor; working of the Heartbeat Sensor and Arduino based Heart Rate Monitoring System using a practical heartbeat Sensor.

CHAPTER 1
INTRODUCTION

Monitoring heart rate is very important for athletes, patients as it determines the condition of the heart (just heart rate). There are many ways to measure heart rate and the most precise one is using an Electrocardiography.

But the more easy way to monitor the heart rate is to use a Heartbeat Sensor. It comes in different shapes and sizes and allows an instant way to measure the heartbeat.

Heartbeat Sensors are available in Wrist Watches (Smart Watches), Smart Phones, chest straps, etc. The heartbeat is measured in beats per minute or bpm, which indicates the number of times the heart is contracting or expanding in a minute.

1.1 EXISTING SYSTEM

There are several existing methods for building a heartbeat sensor using Arduino. Here are some popular methods:

1. Pulse Sensor (Optical Heartbeat Sensor)

Description: This is a plug-and-play optical heart-rate sensor designed for Arduino. It measures the change in blood volume using light. The sensor consists of an LED that shines light onto the skin and a photodiode that measures the amount of light reflected.

Working Principle: As the heart pumps blood, the volume of blood under the skin changes. The amount of light reflected varies based on the blood volume, which is captured by the sensor.

Code: Arduino code reads analog data from the sensor and processes it to extract heartbeat information (BPM).

2. Photoplethysmography (PPG) Sensor

Description: PPG sensors are commonly used in smartwatches and fitness trackers. They measure heart rate by detecting blood flow changes using infrared light.

Working Principle: Similar to the pulse sensor, it uses light to detect blood flow changes.

Code: Arduino libraries (such as the "MAX30100" or "MAX30102" libraries) provide functions for detecting heart rate, oxygen levels, and more.

1.2 PROPOSED SYSTEM

System Architecture:

Heart Beat Sensor: The heart beat sensor detects the blood flow variations through the finger or earlobe, which corresponds to heartbeats.

Arduino Microcontroller: The Arduino collects the signals from the sensor, processes them, and calculates the heart rate (in beats per minute, BPM).

Display: The processed heart rate is displayed on an LCD/OLED screen.

Power Supply: The system is powered by USB or battery

Steps to Build:

Connect the Heartbeat Sensor:

Sensor VCC to Arduino 5V.

Sensor GND to Arduino GND.

Sensor Signal pin to Arduino A0.

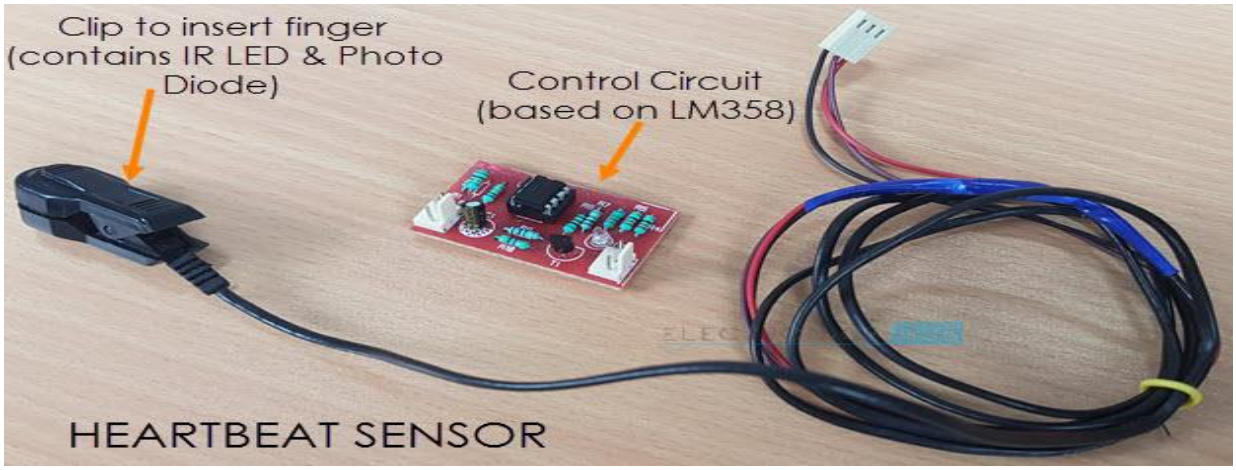
Connect the LCD: Follow standard wiring for the LCD based on its type (e.g., connect RS, E, D4-D7 pins to appropriate digital pins on the Arduino).

Power the System: Provide power to the Arduino either via USB or battery pack.

Upload the Code: Write and upload the Arduino code using the Arduino IDE.

Testing: Place the heart beat sensor on your finger or earlobe, and the heart rate will be displayed on the screen.

1.3 PRINCIPLE OF HEART BEAT SENSOR



The principle behind the working of the Heartbeat Sensor is Photoplethysmograph. According to this principle, the changes in the volume of blood in an organ is measured by the changes in the intensity of the light passing through that organ.

Usually, the source of light in a heartbeat sensor would be an IR LED and the detector would be any Photo Detector like a Photo Diode, an LDR (Light Dependent Resistor) or a Photo Transistor.

With these two i.e. a light source and a detector, we can arrange them in two ways: A Transmissive Sensor and a Reflective Sensor.

In a Transmissive Sensor, the light source and the detector are place facing each other and the finger of the person must be placed in between the transmitter and receiver.

Reflective Sensor, on the other hand, has the light source and the detector adjacent to each other and the finger of the person must be placed in front of the sensor.

CHAPTER 2
BLOCK DIAGRAM AND ITS DESCRIPTION

2.1 BLOCK DIAGRAM

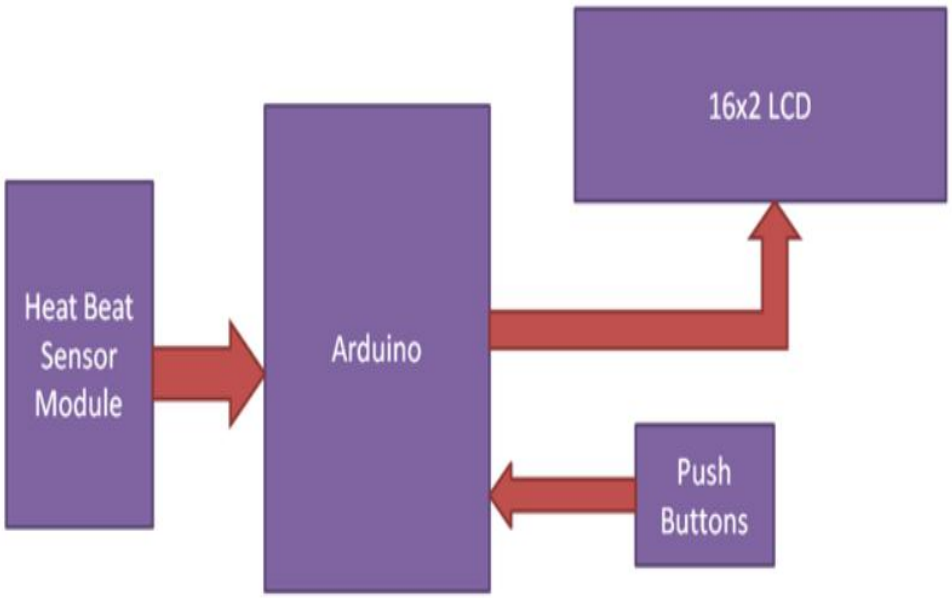


Fig 2.1

2.2 DISCRIPTION

In this project we have used **Heart beat sensor module** to detect Heart Beat. This sensor module contains an IR pair which actually detect heart beat from blood. Heart pumps the blood in body which is called heart beat, when it happens the blood concentration in body changes. And we use this change to make a voltage or pulse electrically.

Heart Beat Sensor: The heart beat sensor detects the blood flow variations through the finger or earlobe, which corresponds to heartbeats.

Arduino Microcontroller: The Arduino collects the signals from the sensor, processes them, and calculates the heart rate (in beats per minute, BPM).

Display: The processed heart rate is displayed on an LCD/OLED screen.

Power Supply: The system is powered by USB or battery

2.2.1 ARDUINO MICROCONTROLLER

Arduino is an open-source platform used for building electronics projects. Arduino consists of both a physical programmable circuit board (often referred to as a microcontroller) and a piece of software, or IDE (Integrated Development Environment) that runs on your computer, used to write and upload computer code to the physical board.

The Arduino platform has become quite popular with people just starting out with electronics, and for good reason. Unlike most previous programmable circuit boards, the Arduino does not need a separate piece of hardware (called a programmer) in order to load new code onto the board -- you can simply use a USB cable. Additionally, the Arduino IDE uses a simplified version of C++, making it easier to learn to program. Finally, Arduino provides a standard form factor that breaks out the functions of the micro-controller into a more accessible package.

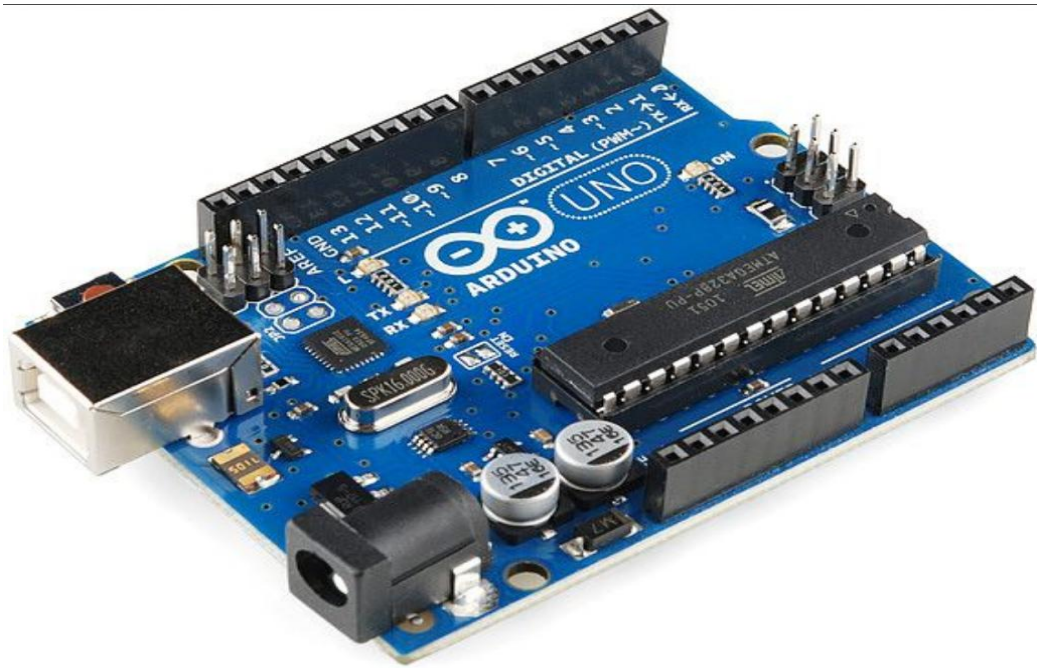


Fig 2.2.1

ARDUINO UNO PIN CONFIGURATION:

Analog Pins: The pins A0 to A5 are used as an analog input and it is in the range of 0-5v

Digital Pins: The pins 0 to 13 are used as a digital input or output for the Arduino board.

AREF Pin: This is an analog reference pin of the Arduino board. It is used to provide a reference voltage from an external power supply.

SPI Pins: This is the Serial Peripheral Interface pin, it is used to maintain SPI communication with the help of the SPI library. SPI pins include:

SS: Pin number 10 is used as a Slave Select

MOSI: Pin number 11 is used as a Master Out Slave In

MISO: Pin number 12 is used as a Master In Slave Out

SCK: Pin number 13 is used as a Serial Clock

Serial Pins: These pins are also known as a UART pin. It is used for communication between the Arduino board and a computer or other devices. The transmitter pin number 1 and receiver pin number 0 is used to transmit and receive the data resp.

External Interrupt Pins: This pin of the Arduino board is used to produce the External interrupt and it is done by pin numbers 2 and 3.

PWM Pins: This pins of the board is used to convert the digital signal into an analog by varying the width of the Pulse. The pin numbers 3,5,6,9,10 and 11 are used as a PWM pin.

Reset: This pin of the board is used to reset the microcontroller. It is used to Resets the microcontroller.

LED Pin: The board has an inbuilt LED using digital pin-13. The LED glows only when the digital pin becomes high.

2.2.2 16X2 LCD DISPLAY

Fig 2.2.2

- **VSS (Pin 1)**: This is the ground pin. It should be connected to the ground of the system.
- **DD (Pin 2)**: This is the power supply pin for the LCD, usually 5V.
- **VO (Pin 3)**: This pin is used to adjust the contrast of the display. You can connect it to a potentiometer to vary the contrast.

- **RS (Pin 4)**: This pin selects the **Register**. When **RS = 0**, the command register is selected (for instructions like clearing the display). When **RS = 1**, the data register is selected (for writing characters to the display).
- **RW (Pin 5)**: This pin selects the mode: **Read/Write**. If **RW = 0**, the LCD is in write mode (data is sent from the microcontroller to the LCD). This pin is usually connected to ground to keep the LCD in write mode.
- **E (Pin 6)**: This is the **Enable** pin. A high-to-low transition on this pin tells the LCD that data is available to be read.
- **D0-D7 (Pins 7 to 14)**: These are the data pins. In **8-bit mode**, all 8 data pins (D0-D7) are used. In **4-bit mode**, only D4-D7 are used, and data is sent in two sets (nibbles).
- **LED+ (Pin 15)**: This is the **anode** (+) pin of the backlight LED. It is usually connected to 5V.
- **LED- (Pin 16)**: This is the **cathode** (-) pin of the backlight LED. It is connected to ground.

2.2.3 HEART BEAT SENSOR MODULE

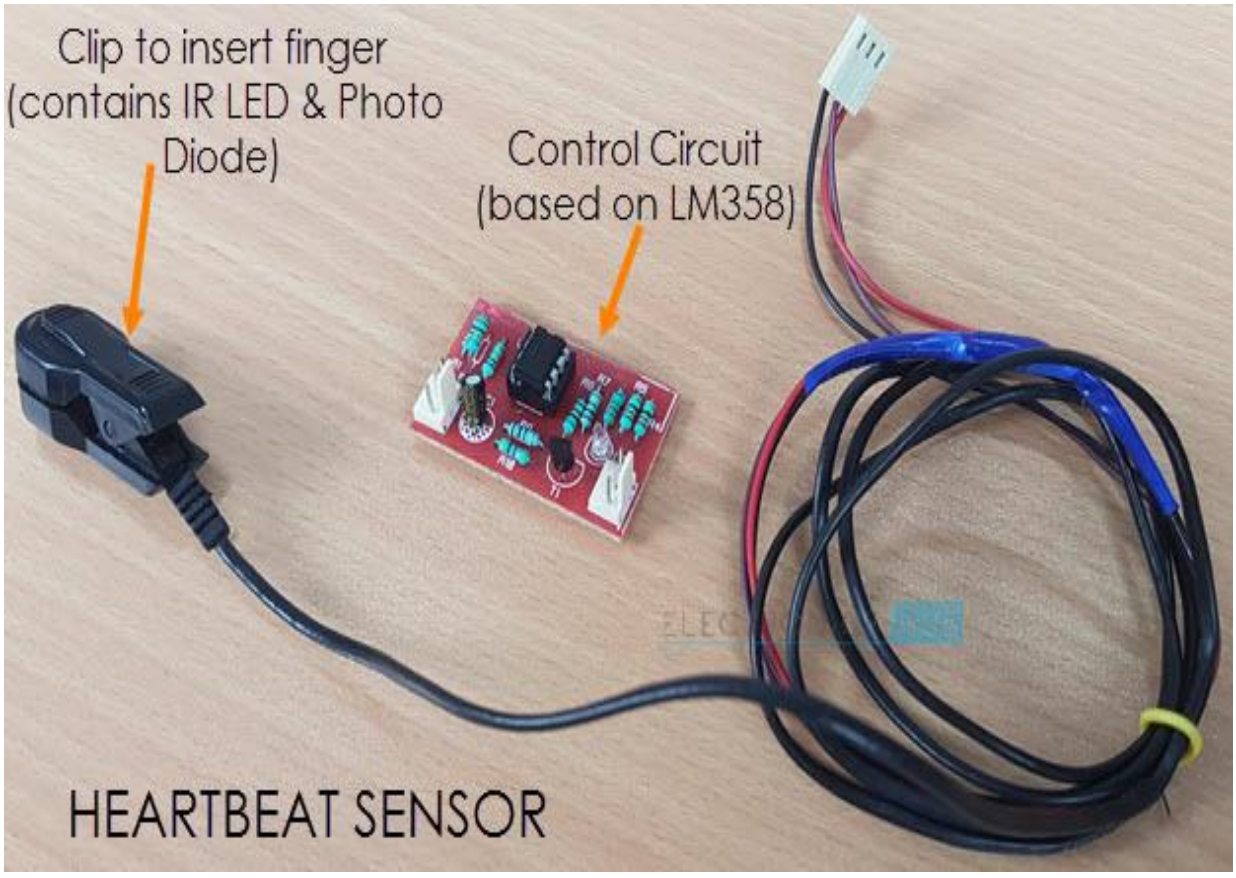


Fig 2.2.3

A typical **heart beat sensor module** (like the popular KY-039 or Pulse Sensor) used with Arduino has **3 pins**.

Below is the pin configuration and description for the heart beat sensor module. **VCC (Power**

Supply Pin): Provides power to the sensor.Typically, the sensor operates at **3.3V** or **5V**, so you can connect this pin to the **5V** or **3.3V** pin of the Arduino, depending on the module.**GND**

(Ground Pin): This pin is connected to the ground of the circuit to complete the power connection.It should be connected to the **GND** pin of the Arduino or another ground reference in your project.

Signal (Analog Output Pin): This pin outputs an **analog signal** that corresponds to the heart beat.The signal is a voltage waveform, where peaks represent heart beats.This pin is connected to one of the **analog input pins** on the Arduino (e.g., **A0**).

2.3 CIRCUIT OF ARDUINO BASED HEART RATE MONITOR

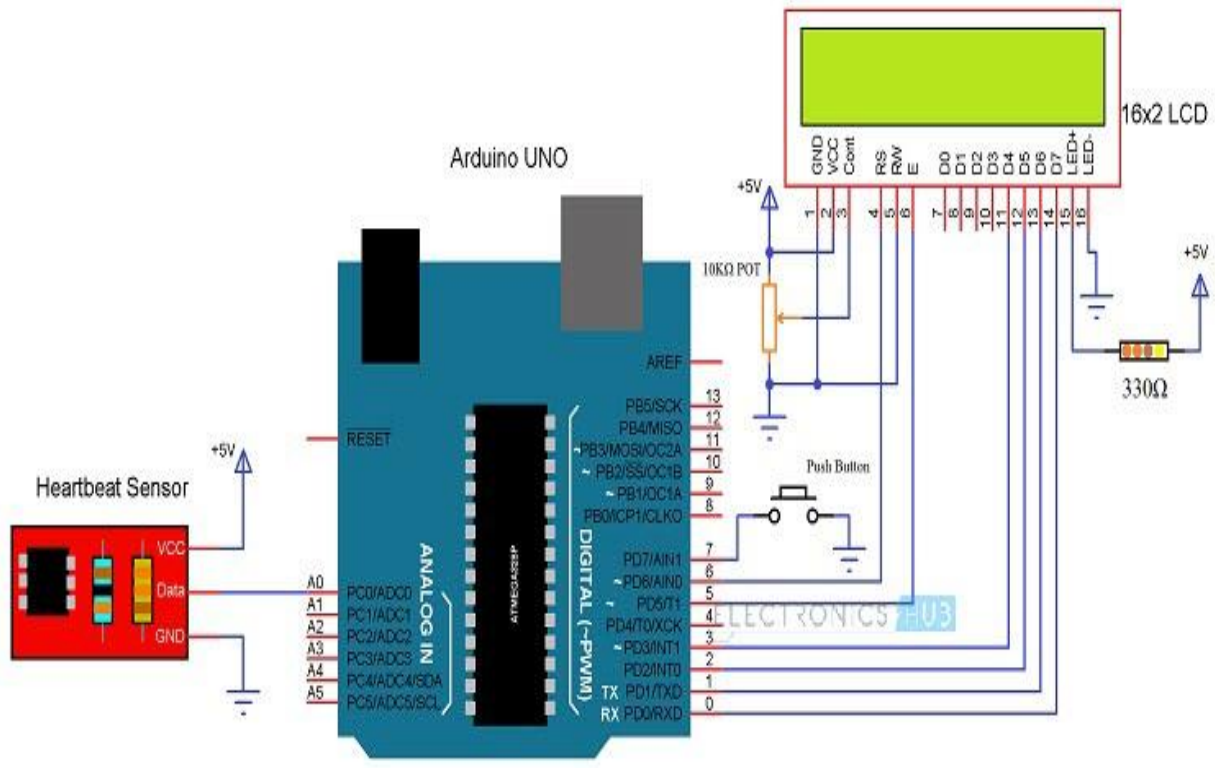


Fig 2.3

The circuit design of Arduino based Heart rate monitor system using Heart beat Sensor is very simple. First, in order to display the heartbeat readings in bpm, we have to connect a 16×2 LCD Display to the Arduino UNO.

The 4 data pins of the LCD Module (D4, D5, D6 and D7) are connected to Pins 1, 1, 1 and 1 of the Arduino UNO. Also, a 10KΩ Potentiometer is connected to Pin 3 of LCD (contrast adjust pin). The RS and E (Pins 3 and 5) of the LCD are connected to Pins 1 and 1 of the Arduino UNO.

CHAPTER 3

CIRCUIT WORKING & ITS DESCRIPTION

3.1 WORKING OF CIRCUIT

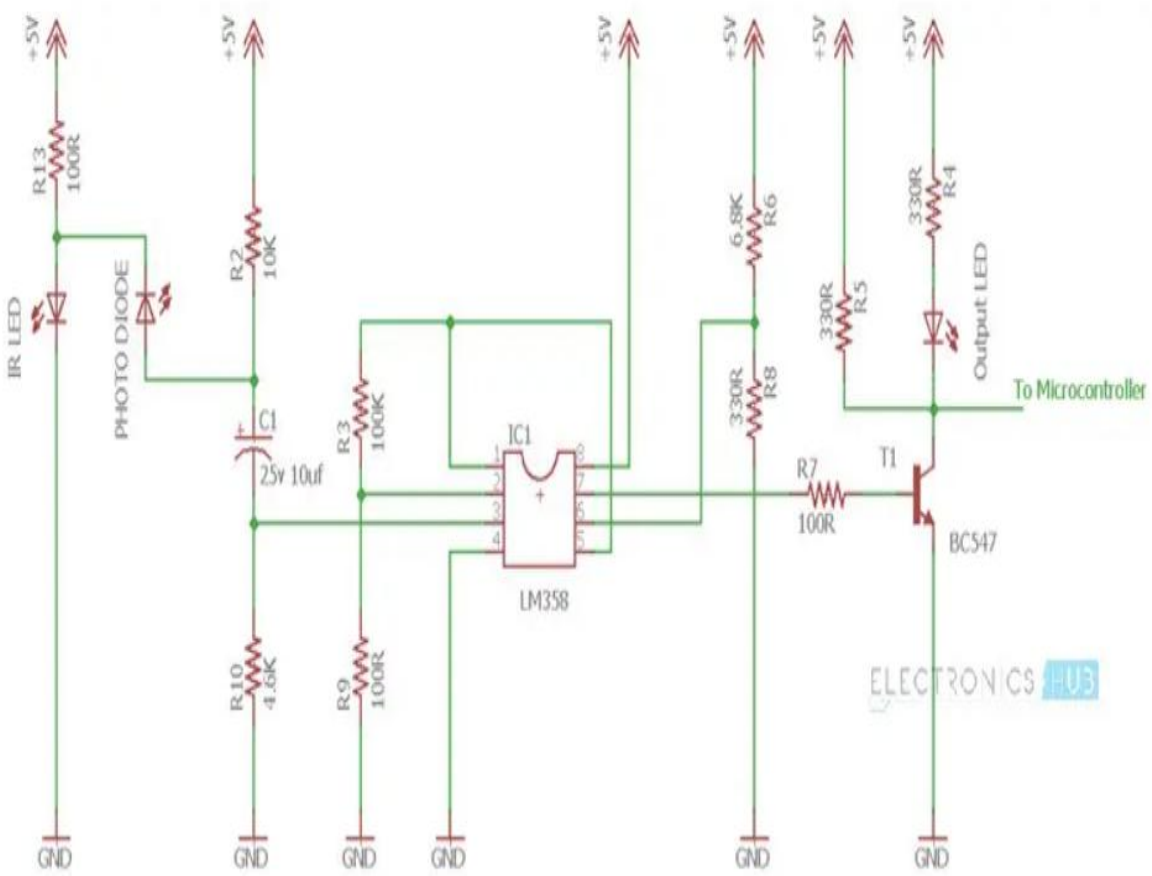


Fig 3.1

A simple Heartbeat Sensor consists of a sensor and a control circuit. The sensor part of the Heartbeat Sensor consists of an IR LED and a Photo Diode placed in a clip.

The Control Circuit consists of an Op-Amp IC and few other components that help in connecting the signal to a Microcontroller. The working of the Heartbeat Sensor can be understood better if we take a look at its circuit diagram.

The above circuit shows the finger type heartbeat sensor, which works by detecting the pulses. Every heartbeat will alter the amount of blood in the finger and the light from the IR LED passing through the finger and thus detected by the Photo Diode will also vary.

The output of the photo diode is given to the non – inverting input of the first op – amp through a capacitor, which blocks the DC Components of the signal. The first op – amp cats as a non – inverting amplifier with an amplification factor of 1001.

The output of the first op – amp is given as one of the inputs to the second op – amp, which acts as a comparator. The output of the second op – amp triggers a transistor, from which, the signal is given to a Microcontroller like Arduino.

The Op – amp used in this circuit is LM358. It has two op – amps on the same chip. Also, the transistor used is a BC547. An LED, which is connected to transistor, will blink when the pulse is detected.

CHAPTER 4

SOFTWARE DEVELOPMENT

4.1 Software Used

The software development of a **heart beat sensor using Arduino** involves writing a program to read the sensor's data, process it, and display the heart rate in **beats per minute (BPM)**. Here's a step-by-step guide, including the code for interfacing the heart beat sensor with Arduino.

ARDUINO IDE:

The Arduino Integrated Development Environment (IDE) is the software platform used for writing, compiling, and uploading code to Arduino boards. It is designed to be user-friendly, making it accessible for beginners, hobbyists, and even professionals who want to create electronics projects with ease.

The Arduino IDE (Integrated Development Environment) is an open-source software application that allows users to write, compile, and upload code to Arduino boards. It is designed to be beginner-friendly, making it easy for users to develop, debug, and test their projects.

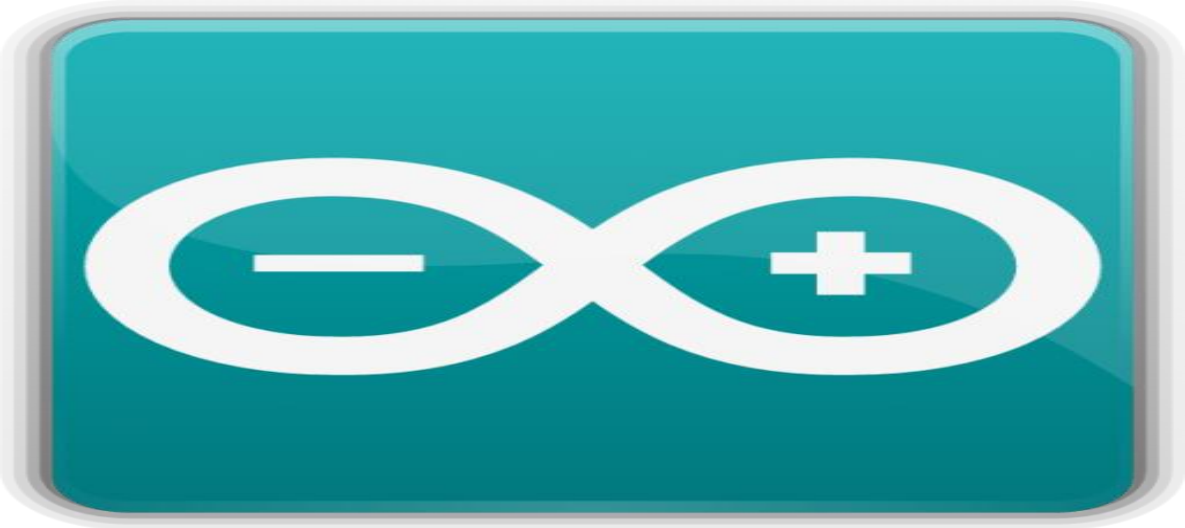


Fig 4.1

- **Steps for Software Development:**

- **Connect the Heart Beat Sensor**:

1.Make sure the sensor's **VCC**, **GND**, and **Signal** pins are properly connected to the Arduino as per the pin configuration.

2.The **Signal** pin is connected to an **analog input pin** of the Arduino (e.g., A0).

❖ **Write the Arduino Code**:

1.The code will read the sensor's analog output, detect peaks (representing heart beats), and calculate the time between peaks to compute the heart rate in BPM.

2.The result is displayed on the **Serial Monitor** or optionally on an **LCD**.

4.2 REQUIREMENTS

- Arduino UNO
- 16X2 LCD Display
- Push Button
- 10K ohm Potentiometer
- Heart Beat Sensor Module
- Connecting Wires
- Bread Board
- USB Cable
- Arduino IDE
- 330 ohm Resistor(optional- for LCD back light)

CHAPTER 5

RESULTS

5.1 SCREENSHOTS WITH DETAILED EXPLAINATION

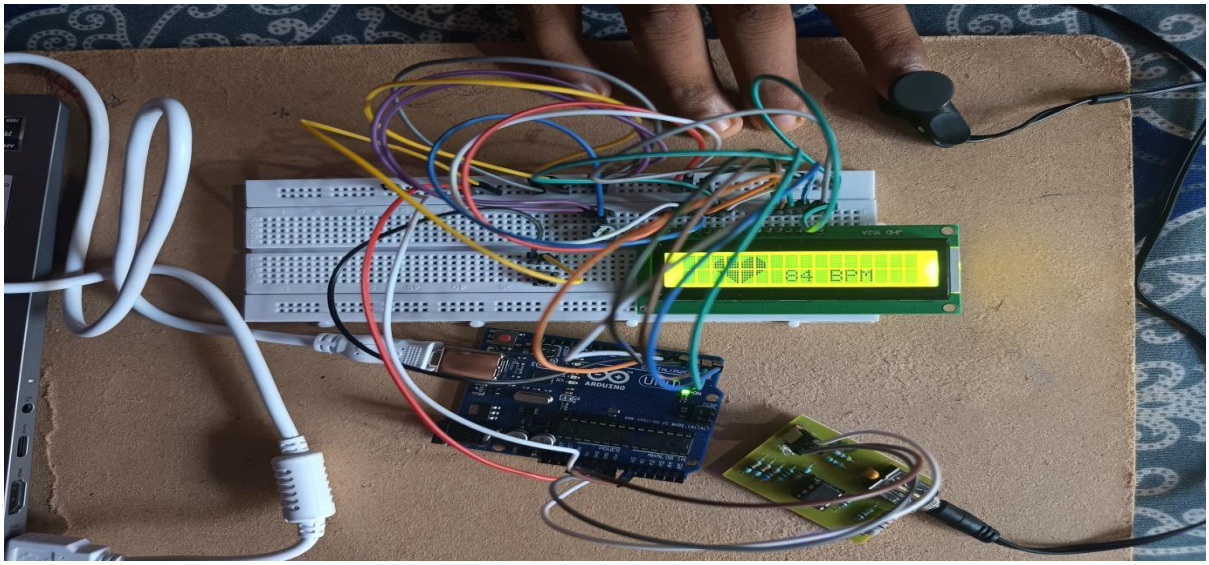


Fig 5.1

Upload the code to Arduino UNO and Power on the system. The Arduino asks us to place our finger in the sensor and press the switch.

Place any finger (except the Thumb) in the sensor clip and push the switch (button). Based on the data from the sensor, Arduino calculates the heart rate and displays the heartbeat in bpm.

While the sensor is collecting the data, sit down and relax and do not shake the wire as it might result in a faulty values.

After the result is displayed on the LCD, if you want to perform another test, just push the rest button on the Arduino and start the procedure once

CHAPTER 6

CONCLUSIONS & FUTURE SCOPE

6.1 ADVANTAGES

1. Cost-Effective

Arduino and heartbeat sensors are relatively inexpensive, making it affordable for DIY enthusiasts and educational purposes.

2. Easy to Interface

Heartbeat sensors are easy to interface with Arduino. Arduino provides simple libraries to read the sensor data and process it.

3. Real-Time Monitoring

You can monitor heart rate in real-time and display it on various outputs like LCDs, serial monitors, or even transmit the data wirelessly for remote monitoring.

4. Customization

Arduino allows users to customize the project according to their specific needs, whether it's for fitness tracking, healthcare, or educational experiments.

5. Simple Integration with Other Sensors

Arduino can easily integrate multiple sensors (e.g., temperature, oxygen levels) along with the heartbeat sensor for more comprehensive health monitoring.

6. Open-Source Libraries

A wide range of open-source libraries and code examples are available, making it easier for beginners to work with heartbeat sensors and Arduino without starting from scratch.

7. Wide Range of Applications

Heartbeat sensors with Arduino are useful in various fields:

Healthcare: Home health monitoring, fitness trackers, and telemedicine.

Education: Teaching electronics, sensors, and health-related technology.

Wearable Devices: For fitness and health-conscious people.

8. Low Power Consumption

Arduino-based heartbeat monitoring systems can be optimized for low power consumption, making them suitable for battery-operated or portable systems.

By combining an Arduino and a heartbeat sensor, you can create a flexible, real-time heart rate monitoring system at a fraction of the cost of commercial devices.

6.2 DISADVANTAGES

Limited Accuracy: The accuracy of heartbeat sensors like the pulse sensor or photoelectric pulse oximeters can be influenced by movement, ambient light, and sensor placement, leading to inaccurate readings.

Noise and Interference: Sensors can be prone to noise from electrical interference, poor contact with the skin, or irregular heart rhythms, which can lead to false or fluctuating readings.

Requires Calibration: Heartbeat sensors often need to be calibrated to the individual's specific body and environmental conditions. Without proper calibration, the readings may not be reliable.

Environmental Sensitivity: Ambient light, temperature, and even skin color can affect the performance of optical sensors, reducing the reliability of the heartbeat measurement.

Limited Processing Power: Arduino boards have limited computational power and memory. If additional complex signal processing (like noise filtering or signal averaging) is required to make readings more accurate, it might strain the Arduino's capabilities.

Power Consumption: Continuous heartbeat monitoring can consume a significant amount of power, especially when combined with other components like displays or wireless modules. This can reduce the battery life of portable systems.

6.3 APPLICATIONS

- A simple project involving Arduino UNO, 16×2 LCD and Heartbeat Sensor Module is designed here which can calculate the heart rate of a person.
- This project can be used as an inexpensive alternative to Smart Watches and other expensive Heart Rate Monitors.

1. Health Monitoring Systems

- **Personal Health Tracking**: Individuals can use a heartbeat sensor with Arduino to track their own heart rate. This data can be stored or uploaded to a cloud-based health app for long-term analysis.
- **Wearable Health Devices**: Arduino can be used to prototype wearable devices like smartwatches or fitness bands that track heart rate continuously during daily activities or workouts.

2. Fitness and Sports Applications

- **Fitness Tracking Systems**: Heart rate monitors can help athletes and fitness enthusiasts track their pulse during exercise. The Arduino can calculate the number of beats per minute (BPM) and display the result on an LCD screen or a mobile app.

3. Heart Rate Alerts and Alarms

- **Heart Rate Alert System**: The Arduino can be programmed to trigger an alarm when the heart rate goes above or below a certain threshold, indicating abnormal activity. This is helpful for patients with heart conditions, allowing caregivers to respond to emergencies quickly.

5. Heart Rate Controlled Systems

- **Heart Rate Controlled Devices**: The Arduino can be used to control devices based on the heart rate. For example, an exercise machine's resistance could automatically adjust depending on the user's heart rate, offering a more personalized workout.

6. Educational Projects and Research

- **Student Projects**: Heartbeat sensors and Arduino are commonly used in educational environments to teach students about bio-signals and how to develop real-world applications. Students can build projects such as heart rate monitors, alert systems, or IoT health devices.
- **Medical Research Prototyping**: Researchers use Arduino to prototype low-cost medical devices to track heart rate and other physiological parameters for studies related to cardiovascular health.

6.4 FUTURE SCOPE

- Remote Health Monitoring
- Mental Health Monitoring
- Education and Research
- Education and Research
- Emergency Response Systems
- Early Detection of Cardiovascular Diseases

REFERENCES

- Heartbeat Sensor Using Arduino (Heart Rate Monitor)Nov 2017RaviRavi. 2017. "Heartbeat Sensor Using Arduino (Heart Rate Monitor).
- Electronics Hub. November 4, 2017.
- Achten, Juul, and Asker E. Jeukendrup. 2003. “Heart Rate Monitoring: Applications and Limitations.” Sports Medicine 33 (7):517–38.
- Dwivedi, Anand. 2014. "Heart Beat Monitor System." February 18, 2014. Website.” n.d. Accessed May 4, 2018. S. Sali, P. Durge, M. Pokarand N. Kasge, “Microcontroller Based Heart Rate Monitor,” International Journal of Science and Research (IJSR), vol. 5, no. 5, pp. 1169-1172, 2016. (10) Development of a ReflectancePhotoplethysmogram Based Heart Rate Monitoring Device. Available from: Based_Heart_Rate_Monitoring_Device [accessed May 042018].

APPENDIX : SOURCE CODE

```
#include <LiquidCrystal.h>

LiquidCrystal lcd(6, 5, 3, 2, 1, 0);
int data = A0;
int start = 7;
int count = 0;
unsigned long temp = 0;
const int threshold = 100; // Adjust this value based on your sensor

byte customChar1[8] = {0b00000, 0b00000, 0b00011, 0b00111, 0b01111, 0b01111,
0b01111, 0b01111};
byte customChar2[8] = {0b00000, 0b11000, 0b11100, 0b11110, 0b11111, 0b11111,
0b11111, 0b11111};
byte customChar3[8] = {0b00000, 0b00011, 0b00111, 0b01111, 0b11111, 0b11111,
0b11111, 0b11111};
byte customChar4[8] = {0b00000, 0b10000, 0b11000, 0b11100, 0b11110, 0b11110,
0b11110, 0b11110};
byte customChar5[8] = {0b00111, 0b00011, 0b00001, 0b00000, 0b00000, 0b00000,
0b00000, 0b00000};
byte customChar6[8] = {0b11111, 0b11111, 0b11111, 0b11111, 0b01111, 0b00111,
0b00011, 0b00001};
byte customChar7[8] = {0b11111, 0b11111, 0b11111, 0b11111, 0b11110, 0b11100,
0b11000, 0b10000};
byte customChar8[8] = {0b11100, 0b11000, 0b10000, 0b00000, 0b00000, 0b00000,
0b00000, 0b00000};

void setup() {
  lcd.begin(16, 2);
  lcd.createChar(1, customChar1);
  lcd.createChar(2, customChar2);
  lcd.createChar(3, customChar3);
```

```
  lcd.createChar(4, customChar4);
  lcd.createChar(5, customChar5);
  lcd.createChar(6, customChar6);
  lcd.createChar(7, customChar7);
  lcd.createChar(8, customChar8);

  pinMode(data, INPUT);
  pinMode(start, INPUT_PULLUP);
}

void loop() {
  lcd.setCursor(0, 0);
  lcd.print("Place The Finger");
  lcd.setCursor(0, 1);
  lcd.print("And Press Start");

  // Wait for start button press (debounce)
  while (digitalRead(start) > 0);
  delay(200); // Debounce delay
  while (digitalRead(start) == 0); // Wait for button release

  count = 0; // Reset count
  lcd.clear();
  temp = millis();

  while (millis() < (temp + 10000)) {
    if (analogRead(data) < threshold) {
      count++;
      lcd.setCursor(6, 0);
      lcd.write(byte(1));
      lcd.setCursor(7, 0);
      lcd.write(byte(2));
```

```
        lcd.setCursor(8, 0);
        lcd.write(byte(3));
        lcd.setCursor(9, 0);
        lcd.write(byte(4));

        lcd.setCursor(6, 1);
        lcd.write(byte(5));
        lcd.setCursor(7, 1);
        lcd.write(byte(6));
        lcd.setCursor(8, 1);
        lcd.write(byte(7));
        lcd.setCursor(9, 1);
        lcd.write(byte(8));

        // Wait for finger removal
        while (analogRead(data) < threshold);
        delay(200); // Small delay to prevent rapid counts
        lcd.clear();
    }
}

lcd.clear();
lcd.setCursor(0, 0);
count = count * 6; // Convert to BPM
lcd.setCursor(2, 0);
lcd.write(byte(1));
lcd.setCursor(3, 0);
lcd.write(byte(2));
lcd.setCursor(4, 0);
lcd.write(byte(3));
lcd.setCursor(5, 0);
lcd.write(byte(4));
```

```
  lcd.setCursor(2, 1);
  lcd.write(byte(5));
  lcd.setCursor(3, 1);
  lcd.write(byte(6));
  lcd.setCursor(4, 1);
  lcd.write(byte(7));
  lcd.setCursor(5, 1);
  lcd.write(byte(8));
  lcd.setCursor(7, 1);
  lcd.print(count);
  lcd.print(" BPM");

  while (1); // Stop further processing
}
```